



Semester Project Report

SLR(1) Parser

Name: Saif Majid Khan

SAP-ID: 57114

Subject: Compiler Construction

Section: CS6-1

Instructor: Sir. Tajammul Shahzad

Department of Computer Science

Riphaah International University

Table of Contents

1. Introduction	2
2. Objectives.....	3
3. Tools and Technologies.....	3
4. System Architecture	4
5. Implementation.....	4
5.1 Grammar Processing	5
5.2 FIRST and FOLLOW Sets.....	5
FIRST Set.....	5
FOLLOW Set	5
5.3 DFA Construction	6
5.4 SLR Parsing Table	6
ACTION Table	6
GOTO Table.....	7
5.5 String Parsing	7
5.6 GUI and Visualization	7
DFA Features.....	8
6. Key Features	8
7. Challenges Faced	9
7.1 DFA Edge Overlapping.....	9
7.2 DFA Zooming.....	9
7.3 Parsing Table Alignment	10
8. Output Screenshots.....	10
8.1 FIRST and FOLLOW Sets.....	11
8.2 DFA Diagram	11
8.3 Parsing Table.....	11
8.4 Parse String Simulation	11
9. GitHub Repository	11
10. Conclusion.....	11
Riphah International University	2

1. Introduction

Parsing is one of the most important phases of compiler construction. After lexical analysis, the parser checks whether the input tokens follow the grammar rules of the language or not. Among bottom-up parsing techniques, SLR(1) parsing is widely used because it is efficient and capable of handling many context-free grammars.

This project presents a complete **SLR(1) Parser Simulator** developed using Python and Tkinter. The application allows the user to enter a grammar, automatically generates the augmented grammar, computes FIRST and FOLLOW sets, constructs the LR(0) DFA, creates the SLR parsing table, and simulates string parsing step-by-step.

The main goal of this project is to visually demonstrate how SLR parsing works internally instead of only studying the theory from books. The project provides an interactive GUI with DFA visualization, parsing tables, and parse trace simulation.

2. Objectives

The main objectives of the project are:

- To implement the complete SLR(1) parsing process from scratch.
- To compute FIRST and FOLLOW sets automatically.
- To generate the LR(0) item DFA.
- To create ACTION and GOTO parsing tables.
- To simulate shift-reduce parsing step-by-step.
- To provide an interactive graphical user interface.
- To visualize DFA states and transitions clearly.
- To improve understanding of compiler parsing concepts.

3. Tools and Technologies

Technology	Purpose
Python 3	Main programming language
Tkinter	GUI development
ttk Widgets	Tables and notebook tabs
Canvas	DFA visualization
Python Collections	Data structures like defaultdict
Object-Oriented Programming	Modular system design

4. System Architecture

The project is divided into multiple sections, where each section performs a specific task.

Section	Responsibility
Grammar	Reads and stores productions
FIRST/FOLLOW	Computes FIRST and FOLLOW sets
LR(0) Items & DFA	Generates DFA states and transitions
SLR Table	Creates ACTION and GOTO tables
String Parser	Performs shift-reduce parsing
DFA Diagram	Visualizes states and transitions
GUI	User interaction and display

The workflow of the system is:

Grammar Input → FIRST/FOLLOW → DFA Construction → Parsing Table → String Parsing

5. Implementation

5.1 Grammar Processing

The Grammar section reads user-defined productions and stores:

- Productions
- Non-terminals
- Terminals
- Start symbol

The system automatically augments the grammar by adding:

$S' \rightarrow S$

This augmented grammar is required for SLR parsing.

Example grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

5.2 FIRST and FOLLOW Sets

The project calculates FIRST and FOLLOW sets automatically.

FIRST Set

FIRST tells which terminal can appear first in a derivation.

Example:

$FIRST(F) = \{ id, (\}$

FOLLOW Set

FOLLOW tells which terminal can appear after a non-terminal.

Example:

$\text{FOLLOW}(E) = \{ +,), \$ \}$

These sets are later used while creating reduce entries in the parsing table.

5.3 DFA Construction

The DFA is constructed using LR(0) items.

Each state contains multiple LR(0) items such as:

$E \rightarrow E \bullet + T$

The system performs:

- Closure operation
- GOTO operation
- State generation
- Transition mapping

The DFA is generated automatically from the grammar.

5.4 SLR Parsing Table

After DFA construction, the program generates:

- ACTION table
- GOTO table

ACTION Table

Contains:

- Shift actions
- Reduce actions
- Accept action

GOTO Table

Contains transitions for non-terminals.

The project also detects parsing conflicts such as:

- Shift-Reduce conflict
- Reduce-Reduce conflict

5.5 String Parsing

The parser simulates SLR parsing step-by-step.

The user enters a string such as:

id + id * id

The parser then performs:

- Shift operations
- Reduce operations
- Accept or Reject decision

Every parsing step is displayed in a table showing:

- Stack
- Remaining input
- Action performed

5.6 GUI and Visualization

The graphical interface was built using Tkinter.

The interface contains multiple tabs:

- FIRST/FOLLOW Sets
- DFA Diagram
- Parsing Table
- Parse String

DFA Features

The DFA panel supports:

- Zoom in / Zoom out
- Mouse scrolling
- Hover highlighting
- Curved transitions
- Scrollable canvas

Different colors are used for:

- Normal states
- Accept states
- Hovered states
- Transition labels

6. Key Features

- User-defined grammar input
- Automatic grammar augmentation
- FIRST and FOLLOW computation
- LR(0) DFA generation
- ACTION/GOTO table generation
- Shift-reduce parsing simulation
- Interactive GUI
- DFA zoom and hover effects
- Conflict detection
- Step-by-step parse tracing

7. Challenges Faced

7.1 DFA Edge Overlapping

Initially, transition arrows overlapped with state boxes and labels, making the DFA difficult to read.

This issue was solved by:

- Using curved edges
- Adding routing offsets
- Separating forward and backward transitions

7.2 DFA Zooming

The default DFA became difficult to read for large grammars.

To improve visualization:

- Mouse-wheel zooming was added
- Scrollable canvas was implemented
- Hover highlighting was introduced

7.3 Parsing Table Alignment

Large parsing tables caused display and scrolling problems.

This was solved using:

- Scrollbars
- Treeview widgets
- Dynamic column widths

8. Output Screenshots

8.1 FIRST and FOLLOW Sets

SLR(1) Parser — Compiler Construction Project

Grammar Input

Enter grammar below (one rule per line). Example: $E \rightarrow E + T \mid T$

```

E -> E + T | T
T -> T * F | F
F -> ( E ) | id
  
```

Generate Parser **Clear All**

FIRST / FOLLOW **DFA Diagram** **Parsing Table** **Parse String**

FIRST Sets

```

=====
FIRST( E' ) = { (, id }
FIRST( E ) = { (, id }
FIRST( T ) = { (, id }
FIRST( F ) = { (, id }
  
```

FOLLOW Sets

```

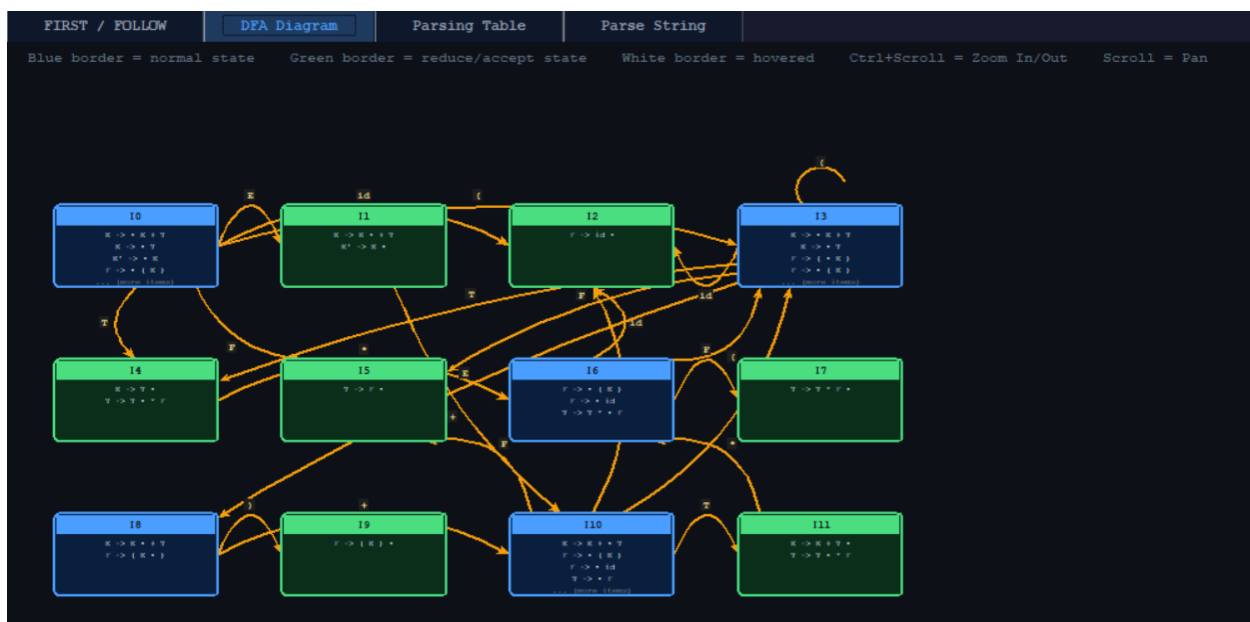
=====
FOLLOW( E' ) = { $ }
FOLLOW( E ) = { $, ), + }
FOLLOW( T ) = { $, ), *, + }
FOLLOW( F ) = { $, ), *, + }
  
```

Success

Parser generated successfully!
States : 12
Productions : 7
Terminals : (,), *, +, id

OK

8.2 DFA Diagram



8.3 Parsing Table

FIRST / FOLLOW	DFA Diagram	Parsing Table	Parse String							
ACTION		GOTO								
State	(	)	*	+	id	\$	E	T	F	
0	s3				s2		1	4	5	
1				s10		acc				
2		r6	r6	r6		r6				
3	s3				s2		8	4	5	
4		r2	s6	r2		r2				
5		r4	r4	r4		r4				
6	s3				s2				7	
7		r3	r3	r3		r3				
8		s9		s10						
9		r5	r5	r5		r5				
10	s3				s2			11	5	
11		r1	s6	r1		r1				

8.4 Parse String Simulation

FIRST / FOLLOW	DFA Diagram	Parsing Table	Parse String
Input String (space-separated tokens): id + id * id Parse String			
ACCEPT			
Step	Stack	Input	Action
1	0	id + id * id \$	Shift id -> go to state 2
2	0 id 2	+ id * id \$	Reduce by F -> id
3	0 F 5	+ id * id \$	Reduce by T -> F
4	0 T 4	+ id * id \$	Reduce by E -> T
5	0 E 1	+ id * id \$	Shift + -> go to state 10
6	0 E 1 + 10	id * id \$	Shift id -> go to state 2
7	0 E 1 + 10 id 2	* id \$	Reduce by F -> id
8	0 E 1 + 10 F 5	* id \$	Reduce by T -> F
9	0 E 1 + 10 T 11	* id \$	Shift * -> go to state 6
10	0 E 1 + 10 T 11 * 6	id \$	Shift id -> go to state 2
11	0 E 1 + 10 T 11 * 6 id 2	\$	Reduce by F -> id
12	0 E 1 + 10 T 11 * 6 F 7	\$	Reduce by T -> T * F
13	0 E 1 + 10 T 11	\$	Reduce by E -> E + T
14	0 E 1	\$	ACCEPT

9. GitHub Repository

GitHub Link:

https://github.com/saif01234567/SLR-1-_PARSER.git

10. Conclusion

This project successfully implements a complete SLR(1) Parser Simulator using Python. The system demonstrates all major compiler parsing concepts including grammar augmentation, FIRST and FOLLOW computation, LR(0) DFA generation, parsing table construction, and shift-reduce parsing.

The project also provides an interactive graphical interface that makes parsing easier to understand visually. Features such as DFA zooming, hover effects, parsing simulation, and conflict detection make the application useful as both a learning tool and an academic project.

Overall, the project helped strengthen understanding of compiler construction concepts and practical parser implementation techniques.